Kernel-Resident Regex and Filesystem Enforcement

DFA-powered eBPF filtering and certificate-safe agent isolation at fleet scale

Justin Ngai
justinngai@meta.com

Meta

Agenda

01 Problem & Motivation

02 Solution Evolution

03 eBPF Regex Engine

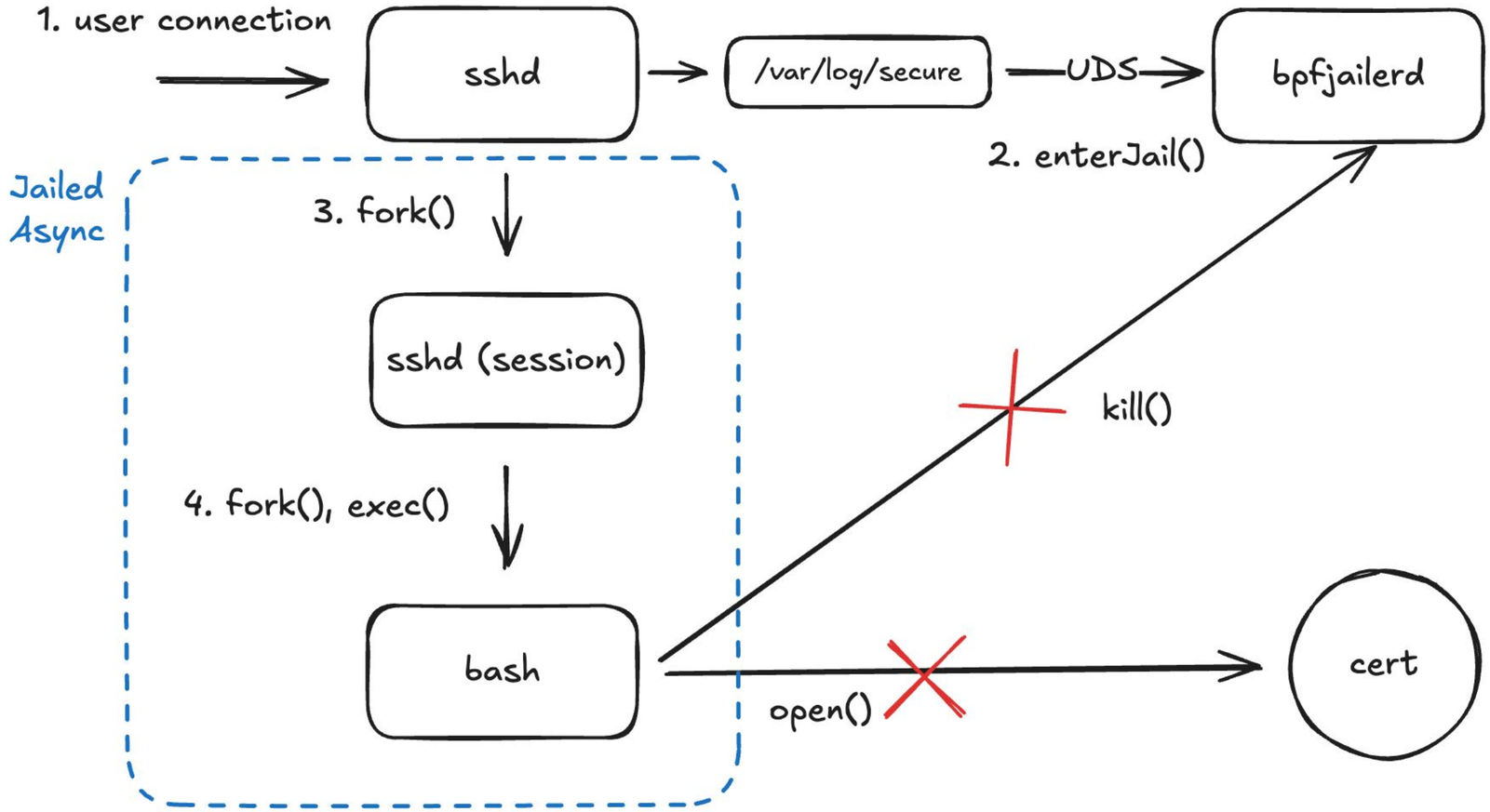
04 Challenges & Lessons Learned

05 Future Work

Certificate Theft & Lateral Movement

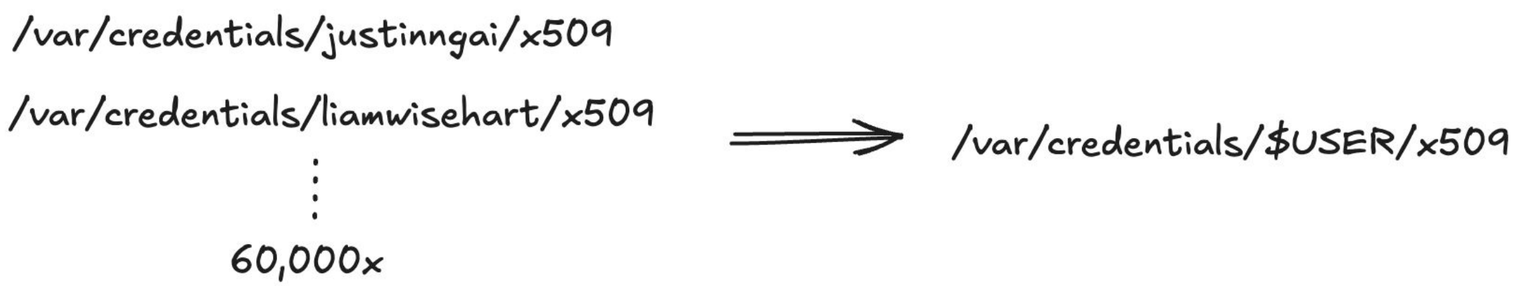
- **Internal red team exercise:** Malicious insider scenario demonstrated:
  - Ability to escalate privileges via impersonation
  - Access to highly sensitive intellectual property (AI model weights, training data)
  - Exfiltration of proprietary assets to external cloud storage
- AI coding assistant proliferation amplifies impersonation risks
- Need to lockdown with various access vectors
- Need to protect certificates (fundamentally just files)

Integrating with SSH (and others)



Challenges

- Needed runtime configurability **but** the FSM for file matching is initialized at startup
  - In some deployment modes this is immutable (without a reboot)
  - Cannot account for workloads that change during the span on the BpfJailer service lifetime
- 100,000 devs = 100,000 separate security policies
- Policy files: ~10MB per service



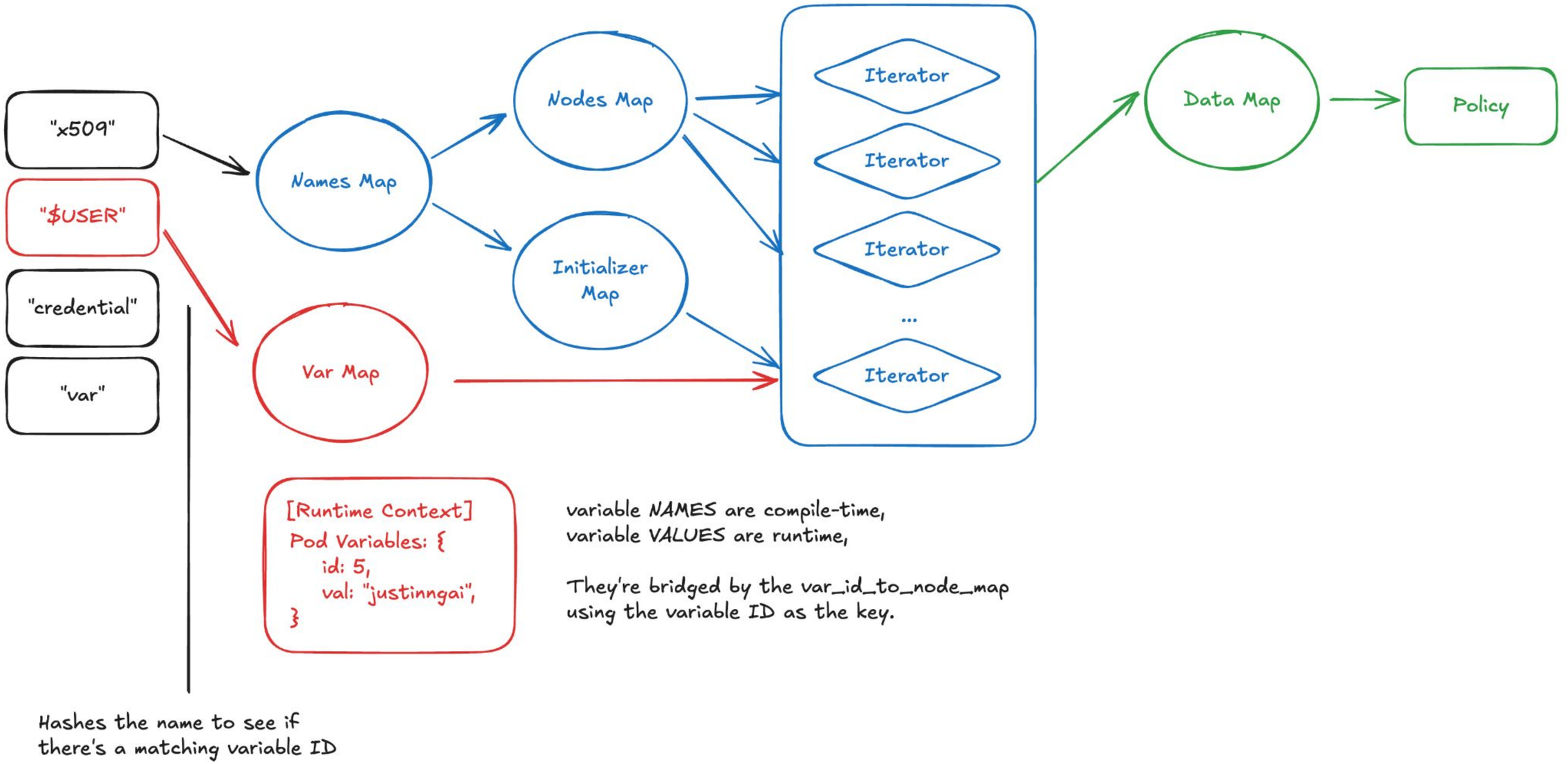
[Compile Time]

"/var/credential/$USER/x509"

↓

"$USER" → Var ID (e.g., 5)

Supporting “Variables” in a Filesystem Jailing



Supporting “Variables” in a Filesystem Jailing

- **Context-aware security policies** using variables instead of hardcoded usernames
- **One policy for all users** e.g. `/home/$USER/` replaces 100,000 static paths
- **Runtime resolution** Variables populated based on SSH cert, VSCode session, container context
  - FSM-based path matching resolves variables at syscall interception time
  - Zero userspace overhead - all enforcement happens in eBPF

Some Limitations

- Current FSM-based file matching only supports whole-component wildcards
  - ✅ Works: /foo/*/bar (match any single directory)
  - ❌ Fails: /var/log/app-*.log (pattern within component)
  - ❌ Fails: /tmp/sess_[0-9a-f]{32} (structured temp files)
  - Example: Blocking rotated logs requires 365+ separate rules

eBPF Regex Engine

Why Regex in eBPF?

- Expressivity ⇒ Character classes (\d, \w), quantifiers ({n,m}), alternation (a|b)
- Performance ⇒ Avoid context switch and system calls overhead
- Security ⇒ Kernel-space enforcement (no TOCTOU, no userspace bypass)
- Glob vs. Regex?

eBPF Regex Engine: Performance

- Integrates regex pattern matching into kernel space for high-frequency security checks.
  - Eliminates 5-50 µs overhead for filtering decisions.
    - MetArmor Filemod (~1B/second*)
      99.4% of MetArmor filemod events *that make it to userspace* are dropped with the regex allowlist (12-07-25)
    - MetArmor Proc Exec (~90B/second*)
      99.6% of MetArmor process execution events *that make it to userspace* are dropped in userspace regex match on cmdline args (as of 12-09-25)
  - Closes security gaps missed by simple pattern matching.

*before any filtering

eBPF Regex Engine: Safety

- Verifier-approved: O(n) linear time, bounded iteration (prevents ReDoS attacks).
  - Real-world examples of issues caused by ReDoS vulnerabilities:
  - In 2019, Cloudflare experienced a service disruption lasting approximately 27 minutes due to a ReDoS vulnerability that allowed crafted input to overwhelm regex processing, resulting in significant performance degradation and temporary service outage (source: Cloudflare Incident Report).
  - Stack Overflow was affected by a ReDoS vulnerability in 2016, causing multiple instances of service degradation and temporary outages of up to 34 minutes during peak traffic periods due to inefficient regular expression patterns (source: Stack Overflow Incident Report).

eBPF Regex Engine: The Potential

- New production use-cases:
  - Match domain name in DNS request
    - Exfiltration prevention via DNS domain content.
    - E.g. Base64 encoded subdomain e.g. `exfil.<base64data>.attacker.com`
  - More flexible file path capture than FSM-based matching.
    - Matching within dentry name e.g. `user-[0-9]+`
  - Match cmdline args in exec syscall
    - Detect command injection
    - Block agent exec high-risk CLI cmds (currently, just memcmp on args)

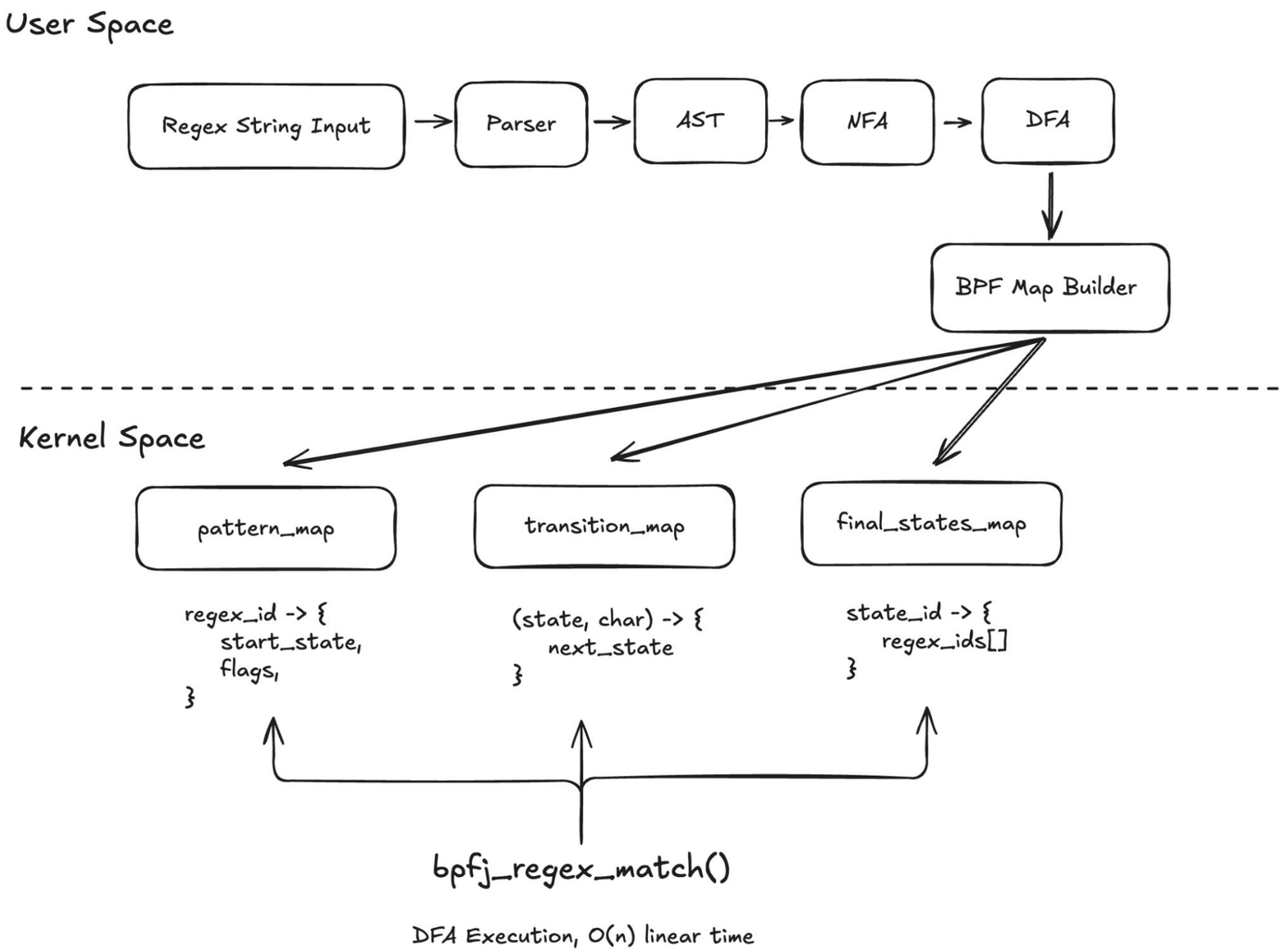
eBPF Regex Engine

⚠️ The Challenge:

- No standard regex library in kernel space
- eBPF verifier constraints: no unbounded loops, no recursion
- Must compile regex to verifier-approved BPF code

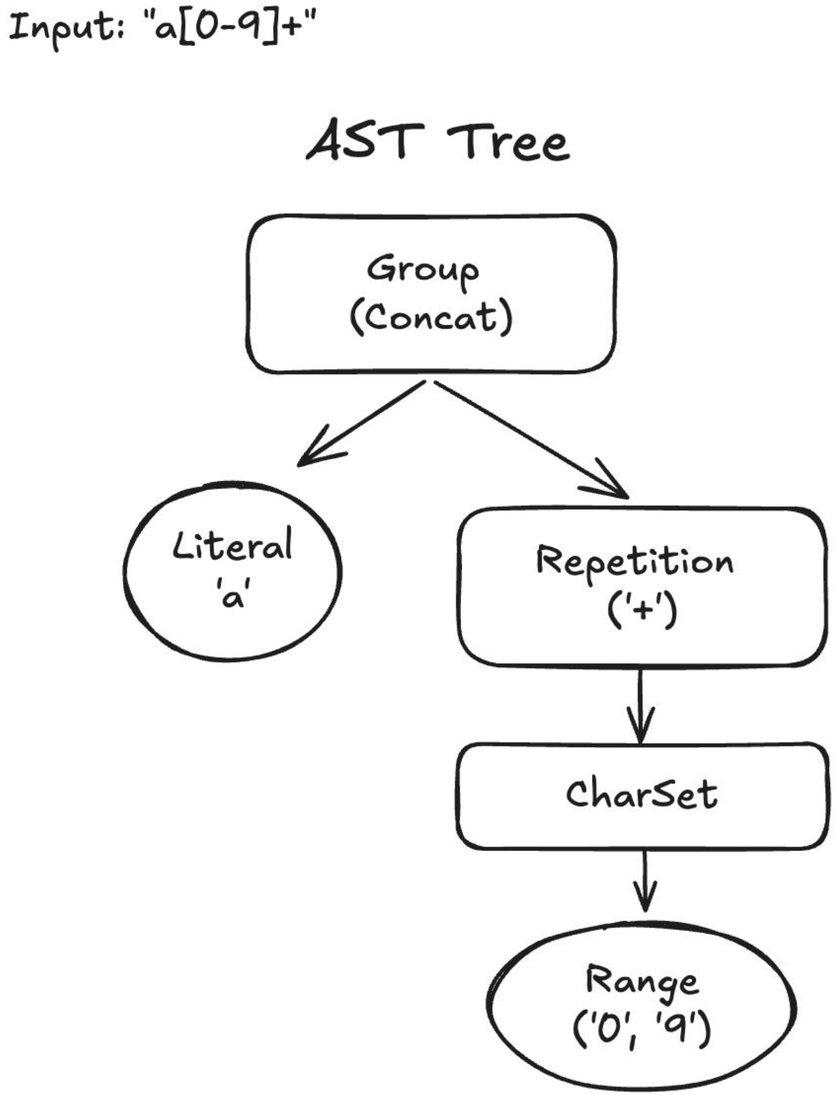
eBPF Regex Engine: High-level Idea

- *“Every regular expression has an equivalent NFA”*
  - Regular Expression Search Algorithm" by Ken Thompson, published in the Communications of the ACM in 1968.
- Parse and compile regex to DFA in userspace (C++)
- Execute DFA in BPF (hash map lookups, bounded iteration)
- Multi-pattern optimization:
  - 8 regexes → 1 DFA → 1 pass



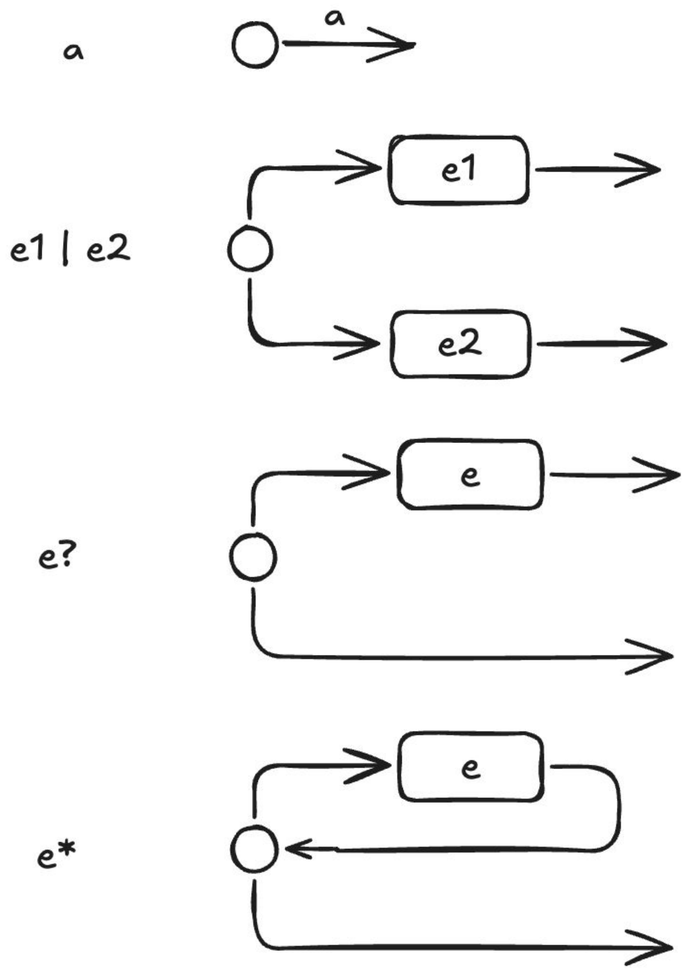
eBPF Regex Engine: Parser → AST

- Parser (Recursive Descent)
- Ex. "a[0-9]+"



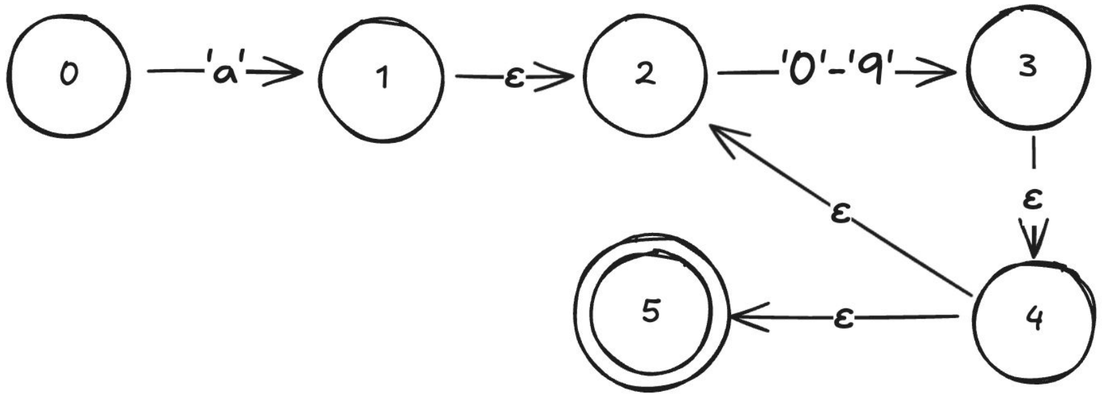
eBPF Regex Engine: AST → NFA (Thompson's Construction)

- The NFA for a regex is built up from partial NFAs for each expression, with a different construction for each operator
- Each AST node generates an NFA fragment:
  - Literal: Single state with character transition
  - CharSet/Range: State with multiple character transitions
  - Repetition: Adds epsilon (ε) loops/bypasses
  - Concat: Connects fragments by linking dangling edges



eBPF Regex Engine: NFA → DFA (Subset Construction)

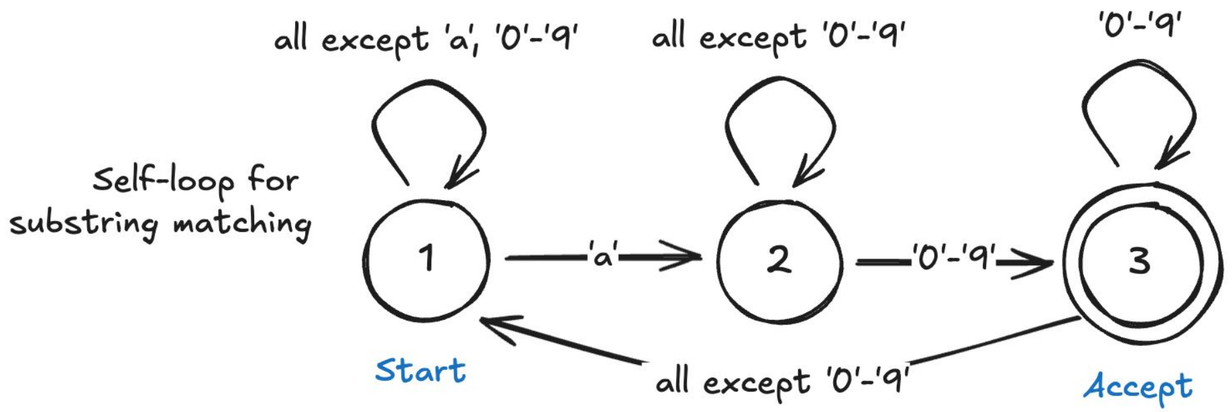
- Subset Construction
  - Iterate over unexplored DFA state and character in alphabet
  - *Add self-loop for substring matching



ε-transitions: 0→1, 1→2, 3→4, 4→5, 4→2 (loop)

Character transitions: 0→1 ('a'), 2→3 ('0'-'9')

eBPF Regex Engine: NFA → DFA (Subset Construction)



State 1: 256 transitions (1 to state 2, 255 self)

State 2: 256 transitions (10 to state 3, 246 to 1)

State 3: 256 transitions (10 self, 246 to state 1)

TOTAL: 768 transitions

eBPF Regex Engine: BPF Runtime Execution

- Byte by byte simulation
  - Any byte is accepted by the algorithm, currently only support ASCII, but can theoretically support support UTF-8 etc.
- For each char in input:
  - Check if state is accepting
  - Lookup (state, char) → next_state
  - Move to next_state
  - Continue or break if dead state

eBPF Regex Engine: DFA → BPF Maps (Serialization)

- The DFA is loaded into 3 BPF hash maps
  - **`patterns_map` - Pattern metadata**
  - `transitions_map` - DFA transition table
  - `final_states_map` - Accepting states

MAP 1: patterns_map

Key	Value

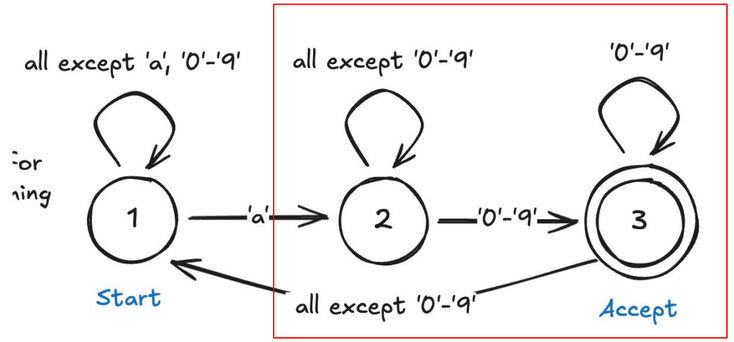
0	bpfj_regex_pattern {
	.regex_id = 0,
	.start_state = 1,
	.flags = 0
	}

```
// Map of regex pattern IDs to pattern metadata
struct bpfj_regex_patterns_map {
 __uint(type, BPF_MAP_TYPE_HASH);
 __uint(max_entries, BPFJ_REGEX_MAX_PATTERNS);
 __type(key, __u32); // regex_id
 __type(value, struct bpfj_regex_pattern);
};
```

eBPF Regex Engine: DFA → BPF Maps (Serialization)

- The DFA is loaded into 3 BPF hash maps
  - `patterns_map` - Pattern metadata
  - **`transitions_map` - DFA transition table**
  - `final_states_map` - Accepting states

```
// DFA state transition key
struct __attribute__((packed)) bpfj_regex_state_char {
 __u32 state_id; // Current DFA state
 __u8 character; // Input character
 ...
// Map of state+character to next state (DFA transition table)
struct bpfj_regex_transitions_map {
 __uint(type, BPF_MAP_TYPE_HASH);
 __uint(max_entries, BPFJ_REGEX_MAX_TRANSITIONS);
 __type(key, struct bpfj_regex_state_char);
 __type(value, struct bpfj_regex_transition);
};
```



MAP 2: transitions_map (512 entries)

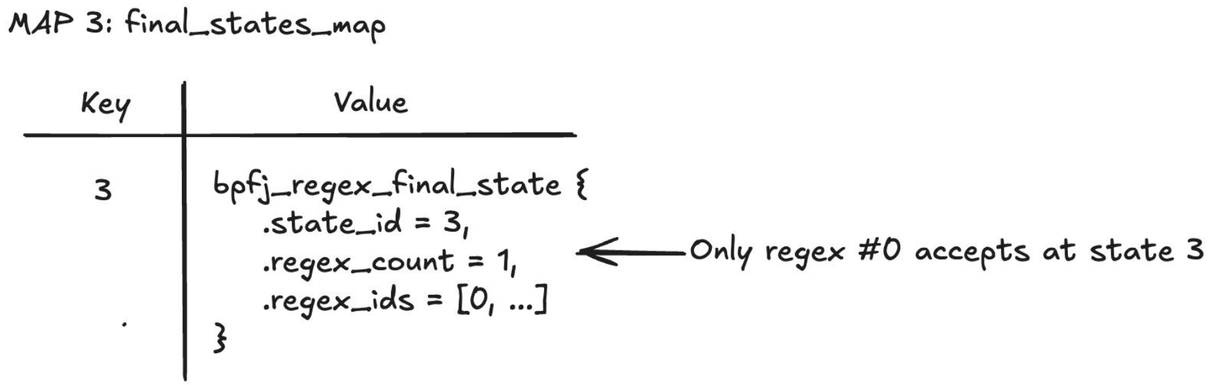
Example for "State 2"

Key	Value	
{state:2, char:0x00}	{next_state:2, regex:0}	← self-loop
{state:2, char:0x01}	{next_state:2, regex:0}	
...	...	
{state:2, char:'/'}	{next_state:2, regex:0}	← self-loop
{state:2, char:'0'}	{next_state:3, regex:0}	← digits
{state:2, char:'1'}	{next_state:3, regex:0}	
...	...	
{state:2, char:'9'}	{next_state:3, regex:0}	
...	...	
{state:2, char:0x00}	{next_state:2, regex:0}	← self-loop

246 self-loops + 10 to state 2 = 256 total

eBPF Regex Engine: DFA → BPF Maps (Serialization)

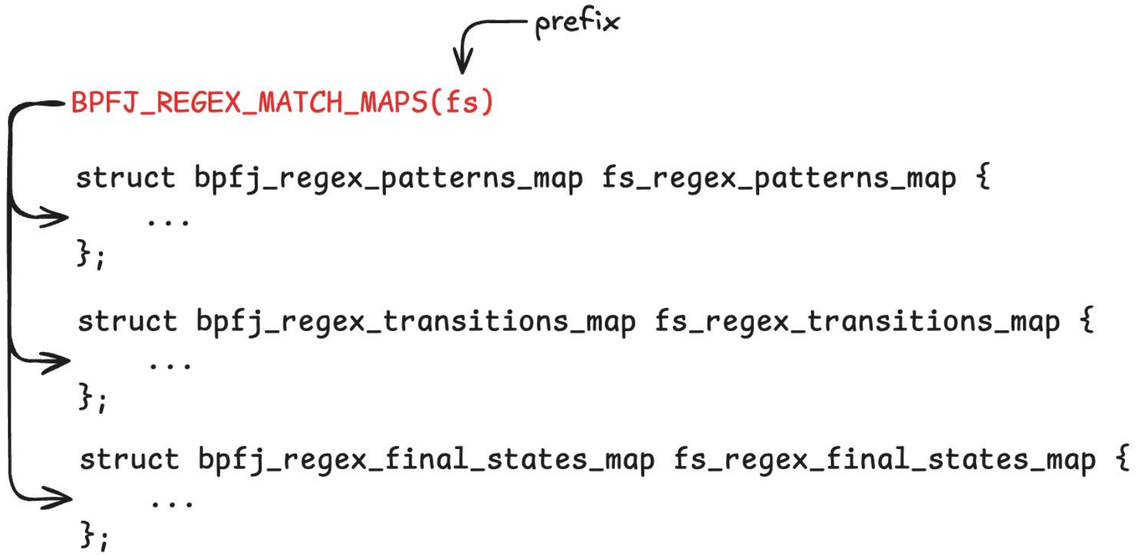
- The DFA is loaded into 3 BPF hash maps
  - `patterns_map` - Pattern metadata
  - `transitions_map` - DFA transition table
  - **`final_states_map` - Accepting states**




```
// Map of final states to their regex IDs
struct bpfj_regex_final_states_map {
 __uint(type, BPF_MAP_TYPE_HASH);
 __uint(max_entries, BPFJ_REGEX_MAX_STATES);
 __type(key, __u32); // state_id
 __type(value, struct bpfj_regex_final_state);
};
```

eBPF Regex Engine: BPF Runtime Setup & Entry Point

1. Declare Maps in BPF Program (Userspace → Kernel)



2. Invoke Matching in LSM Hook

```
SEC("lsm/file_open")
int file_open(struct file *file) {
  char* path = file->f_path.dentry->d_name.name;
  long matches = BPFJ_REGEX_MATCH(fs, path, TEST_STR_LEN);

      (Expands to)
        bpfj_regex_match(path, TEST_STR_LEN,
                         &fs_regex_patterns_map,
                         &fs_regex_transitions_map,
                         &fs_regex_final_states_map);

  if (matches > 0) return -EPERM; // Block on match
}
```

eBPF Regex Engine: Combined DFA Execution (Single-Pass)

```
long bpfj_regex_match(u8* input, u32 len, ...) {
 // Bounds check for verifier
 u32 copy_len = len;
 asm volatile("%0 &= %1" : "+r"(copy_len) : "r"(BPFJ_REGEX_MAX_INPUT_LEN-1));

 // Copy input to per-CPU map
 // ...

 // Setup combined DFA context
 ctx = {
   .input = input_map,
   .transitions_map = transitions_map,
   .final_states_map = final_states_map,
   .start_state = shared_start_state,  // Known at load time
   .current_state = shared_start_state,
   .matched_patterns = matched_patterns,
   .match_count = 0,
 };

 // Execute combined DFA from position 0
 bpfj_regex_combined_dfa_match_from_pos(0, &ctx);

 return ctx.match_count;
}
```

eBPF Regex Engine: Combined DFA Execution (Single-Pass)

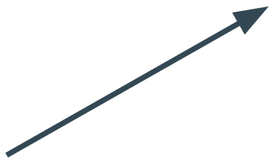
```
long bpfj_regex_match(u8* input, u32 len, ...) {
 // Bounds check for verifier
 u32 copy_len = len;
 asm volatile("%0 &= %1" : "+r"(copy_len) : "r"(BPFJ_REGEX_MAX_INPUT_LEN-1));

 // Copy input to per-CPU map
 // ...

 // Setup combined DFA context
 ctx = {
   .input = input_map,
   .transitions_map = transitions_map,
   .final_states_map = final_states_map,
   .start_state = shared_start_state,  // Known at load time
   .current_state = shared_start_state,
   .matched_patterns = matched_patterns,
   .match_count = 0,
 };

 // Execute combined DFA from position 0
 bpfj_regex_combined_dfa_match_from_pos(0, &ctx);

 return ctx.match_count;
}
```



```
// Single-Pass Matcher (regex_match.h:281)
long combined_dfa_match_from_pos(u32 pos, ctx) {
 ctx->current_state = ctx->start_state;
 ctx->current_pos = pos;

 // Verifier-friendly bounded iteration
 u32 max_steps = ctx->input->len - pos;
 barrier_var(max_steps);  // Prevent optimization
 max_steps &= BPFJ_REGEX_MAX_INPUT_LEN - 1;

 // Use bpf_loop instead of traditional for loop
 bpf_loop(max_steps, &combined_dfa_step, ctx, 0);

 return ctx->error ? 1 : 0;
}
```

eBPF Regex Engine: Combined DFA Execution (Single-Pass)

```
long bpfj_regex_match(u8* input, u32 len, ...) {
 // Bounds check for verifier
 u32 copy_len = len;
 asm volatile("%0 &= %1" : "+r"(copy_len) : "r"(BPFJ_REGEX_MAX_INPUT_LEN-1));

 // Copy input to per-CPU map
 // ...

 // Setup combined DFA context
 ctx = {
   .input = input_map,
   .transitions_map = transitions_map,
   .final_states_map = final_states_map,
   .start_state = shared_start_state,  // Known at load time
   .current_state = shared_start_state,
   .matched_patterns = matched_patterns,
   .match_count = 0,
 };

 // Execute combined DFA from position 0
 bpfj_regex_combined_dfa_match_from_pos(0, &ctx);

 return ctx.match_count;
}
```

```
// Single-Pass Matcher (regex_match.h:281)
long combined_dfa_match_from_pos(u32 pos, ctx) {
 ctx->current_state = ctx->start_state;
 ctx->current_pos = pos;

 // Verifier-friendly bounded iteration
 u32 max_steps = ctx->input->len - pos;
 barrier_var(max_steps);  // Prevent optimization
 max_steps &= BPFJ_REGEX_MAX_INPUT_LEN - 1;

 // Use bpf_loop instead of traditional for loop
 bpf_loop(max_steps, &combined_dfa_step, ctx, 0);

 return ctx->error ? 1 : 0;
}
```

1. Check if accepting (*ctx->final_states_map* lookup)
2. DFA Transition (*ctx->transitions_map* lookup)
3. If dead state, stop DFA walk
4. Else continue

Regex Engine: Challenges

- Combined DFA correctness
  - State space explosion
  - Cross-pattern transition correctness
  - This was a very important feature since, this allowed us to simulate much more efficiently
- Working with State/Transition Limits
  - 64 states, 8192 transitions (# instructions, jump, maps limits)
  - Integration with existing complex programs (MetArmor armors, BpfJailer enforcers) often used within conditional/nested context
  - Pattern simplification required

Literals: a, abc, foo
Character ranges: [a-z], [0-9], [A-Z]
Character classes: \d (digits), \w (word chars), \s (whitespace)
Literal escapes: \n, \t, \r, \f, \v, \a, \\, \., \0
Dot: . matches printable ASCII 32-126
Alternation: a|b, cat|dog, (a|b)*
Repetition: *, +, ?

Regex Engine: Limitations & Next Steps

- Bounds
  - 64 states
  - 8,192 transition limit: issue when DFAs are combined inefficiently
  - Input size: max number of DFA steps per matching attempt
  - Only full string matching for larger strings
- Per-cgroup or per-process regex matchers?
- DFA minimization (Hopcroft) → more expressivity in regex within the above bounds
- Merging the three maps into one → 64 maps limit, support nested map use-case
- Supporting variables in DFA-representations of regex
- Part of our open source plan for 2026

Kernel-Resident Regex and Filesystem Enforcement

DFA-powered eBPF filtering and certificate-safe agent isolation at fleet scale

Justin Ngai
justinngai@meta.com

Meta